LeetCode 78 — Subsets

1. Problem Title & Link

Title: LeetCode 78: Subsets
Link: https://leetcode.com/problems/subsets/

2. Problem Statement (Short Summary)

You are given an array of **unique integers** nums.

Your task:

👉 Return **all possible subsets** (the **power set**) of the array.

📌 Notes:

- The solution **must not contain duplicate subsets**
- The order of subsets does **not matter**
- Empty set [] is also a valid subset

3. Examples (Input → Output)

Example 1

Input:

nums = [1,2,3]

Output:

[
[],
[1],
[2],
[3],
[1,2],
[1,3],
[2,3],
[1,2,3]
]

Example 2

Input:

nums = [0]

Output:

[[], [0]]

4. Constraints

- 1 ≤ nums.length ≤ 10
- All elements are **unique**
- Total subsets = 2^n
- Output size grows exponentially (expected)

5. Core Concept (Pattern / Topic)

Backtracking / Recursion / Bit Manipulation

This is a **classic power set generation** problem.

6. Thought Process (Step-by-Step Explanation)

❌ **Brute Force**

Manually generate combinations → complicated & error-prone.

✔ **Optimized Thinking — Backtracking**

At each element, you have **two choices**:

1. **Include** the element
2. **Exclude** the element

This naturally forms a **decision tree**.

7. Visual / Intuition Diagram (ASCII)

For nums = [1,2]:

```
              []
            /    \
          [1]     []
         /   \    /  \
      [1,2] [1] [2]  []
```

Collected subsets:

[], [1], [2], [1,2]

8. Pseudocode (Language Independent)

```
result = []

function backtrack(index, current):
    if index == n:
        add copy of current to result
        return
```

```
    # include nums[index]
    current.add(nums[index])
    backtrack(index + 1, current)
    current.remove_last()

    # exclude nums[index]
    backtrack(index + 1, current)

backtrack(0, [])
return result
```

9. Code Implementation

✅ **Python (Backtracking)**

```
class Solution:
    def subsets(self, nums):
        res = []
        n = len(nums)

        def backtrack(index, path):
            if index == n:
                res.append(path[:])
                return

            # include
            path.append(nums[index])
            backtrack(index + 1, path)
            path.pop()

            # exclude
            backtrack(index + 1, path)

        backtrack(0, [])
        return res
```

✅ **Java (Backtracking)**

```
import java.util.*;

class Solution {
    public List<List<Integer>> subsets(int[] nums) {
```

```
        List<List<Integer>> res = new ArrayList<>();
        backtrack(0, nums, new ArrayList<>(), res);
        return res;
    }

    private void backtrack(int index, int[] nums, List<Integer> path,
List<List<Integer>> res) {
        if (index == nums.length) {
            res.add(new ArrayList<>(path));
            return;
        }

        // include
        path.add(nums[index]);
        backtrack(index + 1, nums, path, res);
        path.remove(path.size() - 1);

        // exclude
        backtrack(index + 1, nums, path, res);
    }
}
```

10. Time & Space Complexity

Time

- **O(2ⁿ)** — every subset is generated

Space

- **O(n)** recursion depth
- Output space = **O(2ⁿ)** (mandatory)

11. Common Mistakes / Edge Cases

❌ **Mistakes**

- Forgetting to copy the current subset
- Modifying list after adding to result
- Missing empty subset

✔ **Edge Cases**

- Single element array
- Empty input (theoretically → [[]])

12. Detailed Dry Run (Step-by-Step Table)

Input:

nums = [1,2]

Step	index	path	Action
1	0	[]	include 1
2	1	[1]	include 2
3	2	[1,2]	add
4	1	[1]	exclude 2
5	2	[1]	add
6	0	[]	exclude 1
7	1	[]	include 2
8	2	[2]	add
9	1	[]	exclude 2
10	2	[]	add

13. Common Use Cases (Real-Life / Interview)

- Feature selection
- Combination generation
- Power set modeling
- Decision space enumeration

14. Common Traps

- Thinking DP is needed (not required)
- Trying permutations instead of subsets
- Forgetting that order doesn't matter

15. Builds To (Related LeetCode Problems)

- LC 90 — Subsets II (with duplicates)
- LC 77 — Combinations
- LC 46 — Permutations
- LC 39 — Combination Sum

16. Alternate Approaches + Comparison

1️⃣ Bit Masking

For n elements → numbers from 0 to 2^n - 1

```
for mask in range(1 << n):
    subset = []
```

```
for i in range(n):
    if mask & (1 << i):
        subset.append(nums[i])
```

- Clean
- Non-recursive

✔️ 2️⃣ **Backtracking (BEST for teaching)**

- Very intuitive
- Scales to many problems

17. Why This Solution Works (Short Intuition)

Each element independently chooses **present or absent**.
Backtracking explores all such combinations exactly once.

18. Variations / Follow-Up Questions

- Handle duplicates (LC 90)
- Generate subsets of size k
- Streaming subsets
- Iterative solution only